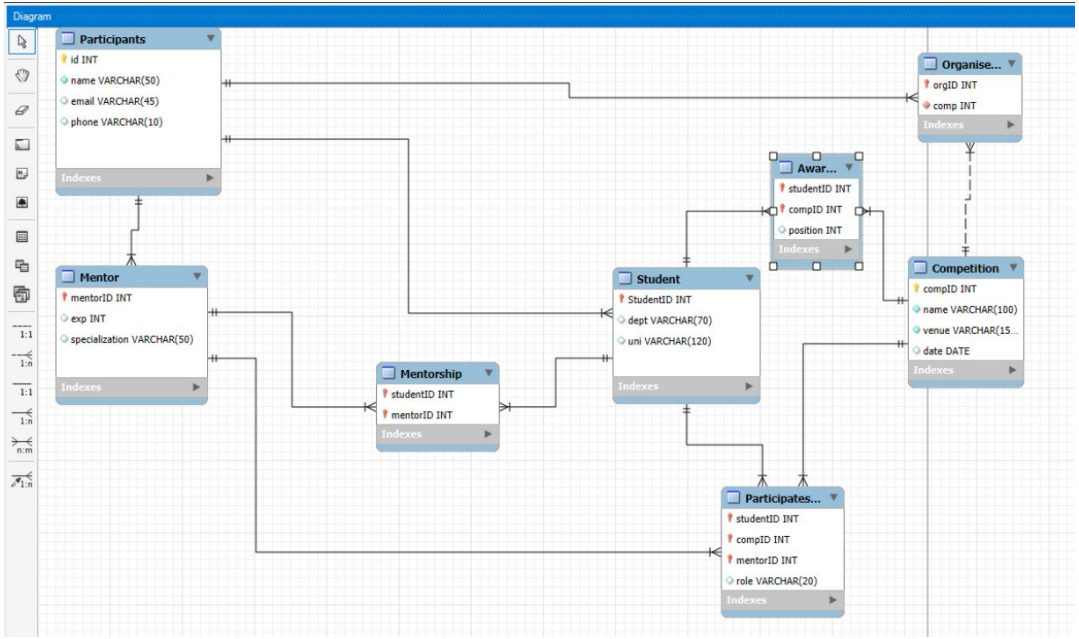
DBMS LabTask5

Submitted By: Patsamatla Bhaavya Ramita, 25MCB1010

Scenario: University Hackathon Database System

This is an implementation of a university database system on using the forward engineering procedure. The scenario chosen is a Hackathon management system, where participants can be students or mentors, competitions are organized, and awards are given. The database includes constraints, superclass-subclass mapping, recursive and ternary relationships, and controlled access through user privileges.



Tables:
Participants,
Student,
Mentor,
Competition,
Organisers,
Mentorship,
MentorHierarchy,
ParticipatesIN, and
Awards.

```sql
CREATE TABLE Participants (
id INT PRIMARY KEY,
    name VARCHAR(50) NOT NULL,
    email VARCHAR(50) UNIQUE,
    phone VARCHAR(10)
);
```

```sql
INSERT INTO Participants (id, name, email, phone) VALUES
(1, 'Arjun', 'arjun1@example.com', '987650001'),
(2, 'Karthik', 'karthik2@example.com', '987650002'),
(3, 'Suresh', 'suresh3@example.com', '987650003'),
(4, 'Manoj', 'manoj4@example.com', '987650004'),
(5, 'Ravi', 'ravi5@example.com', '987650005'),
(6, 'Vignesh', 'vignesh6@example.com', '987650006'),
(7, 'Anand', 'anand7@example.com', '987650007'),
(8, 'Mohan', 'mohan8@example.com', '987650008'),
(9, 'Balaji', 'balaji9@example.com', '987650009'),
(10, 'Krishna', 'krishna10@example.com', '987650010'),
(11, 'Hari', 'hari11@example.com', '987650011'),
(12, 'Praveen', 'praveen12@example.com', '987650012'),
(13, 'Sandhya', 'sandhya13@example.com', '987650013'),
(14, 'Sathish', 'sathish14@example.com', '987650014'),
(15, 'Gopal', 'gopal15@example.com', '987650015'),
(16, 'Ramesh', 'ramesh16@example.com', '987650016'),
(17, 'Vijay', 'vijay17@example.com', '987650017'),
(18, 'Ajay', 'ajay18@example.com', '987650018'),
(19, 'Muthu', 'muthu19@example.com', '987650019'),
(20, 'Aravind', 'aravind20@example.com', '987650020');
```

The Participants table stores the basic details of all individuals in the system. It contains attributes such as **id, name, email, and phone**. Constraints applied include a primary key on id, unique values for email, and the requirement that name cannot be null. This table acts as the **superclass for both students and mentors**.

CREATE TABLE - for creation of table along with constraints and keys.

INSERT INTO - to insert records into the table according to the attributes.

SELECT * FROM table
To display the table's all records.

Result Grid | Filter Rows: | Edit:

id	name	email	phone
1	Arjun	arjun1@example.com	987650001
2	Karthik	karthik2@example.com	987650002
3	Suresh	suresh3@example.com	987650003
4	Manoj	manoj4@example.com	987650004
5	Ravi	ravi5@example.com	987650005
6	Vignesh	vignesh6@example.com	987650006
7	Anand	anand7@example.com	987650007
8	Mohan	mohan8@example.com	987650008
9	Balaji	balaji9@example.com	987650009
10	Krishna	krishna10@example.com	987650010
11	Hari	hari11@example.com	987650011
12	Praveen	praveen12@example.com	987650012
13	Sandhya	sandhya13@example.com	987650013
14	Sathish	sathish14@example.com	987650014
15	Gopal	gopal15@example.com	987650015
16	Ramesh	ramesh16@example.com	987650016
17	Vijay	vijay17@example.com	987650017
18	Ajay	ajay18@example.com	987650018
19	Muthu	muthu19@example.com	987650019
20	Aravind	aravind20@example.com	987650020
NULL	NULL	NULL	NULL

The Student table is a **subclass of Participants**. It includes attributes like **studentID, department, and university**. Each studentID is linked to the Participants table through a foreign key, for proper mapping between a participant and their student details.

```
CREATE TABLE Student (
studentID INT PRIMARY KEY,
dept VARCHAR(70),
    uni VARCHAR(120),
    FOREIGN KEY (studentID) REFERENCES Participants(id) ON DELETE CASCADE
);
```

```
INSERT INTO Student (studentID, dept, uni) VALUES
(1, 'CSE', 'Anna University'),
(2, 'ECE', 'IIT Madras'),
(3, 'EEE', 'Osmania University'),
(4, 'MECH', 'NIT Trichy'),
(5, 'IT', 'VIT Vellore'),
(6, 'Civil', 'SRM University'),
(7, 'CSE', 'BITS Hyderabad'),
(8, 'EEE', 'Andhra University'),
(9, 'ECE', 'JNTU Hyderabad'),
(10, 'MECH', 'PSG Tech');
```

Result Grid | Filter Rows:

studentID	dept	uni
1	CSE	Anna University
2	ECE	IIT Madras
3	EEE	Osmania University
4	MECH	NIT Trichy
5	IT	VIT Vellore
6	Civil	SRM University
7	CSE	BITS Hyderabad
8	EEE	Andhra University
9	ECE	JNTU Hyderabad
10	MECH	PSG Tech
NULL	NULL	NULL

The Mentor table is another **subclass of Participants**. It contains attributes such as **mentorID, experience, and specialization**. A constraint is applied to ensure that the experience value must be greater than zero.

```
CREATE TABLE Mentor (
mentorID INT PRIMARY KEY,
    exp INT CHECK (exp > 0),
    specialization VARCHAR(50),
    FOREIGN KEY (mentorID) REFERENCES Participants(id) ON DELETE CASCADE
);
```

```
INSERT INTO Mentor (mentorID, exp, specialization) VALUES
(11, 5, 'AI'),
(12, 7, 'IoT'),
(13, 3, 'Blockchain'),
(14, 10, 'Embedded'),
(15, 6, 'CyberSecurity'),
(16, 8, 'Cloud'),
(17, 4, 'Data Science'),
(18, 12, 'Robotics'),
(19, 15, 'VLSI'),
(20, 9, 'Networking');
```

Result Grid | Filter Rows:

mentorID	exp	specialization
11	5	AI
12	7	IoT
13	3	Blockchain
14	10	Embedded
15	6	CyberSecurity
16	8	Cloud
17	4	Data Science
18	12	Robotics
19	15	VLSI
20	9	Networking
NULL	NULL	NULL

```sql
CREATE TABLE Competition (
compID INT PRIMARY KEY,
    name VARCHAR(100) UNIQUE NOT NULL,
    venue VARCHAR(150) NOT NULL,
    date DATE DEFAULT (CURRENT_DATE)
);
```

The Competition table stores information about hackathon events. It consists of attributes such as **compID, name, venue, and date**. The table is designed with constraints including a unique name for each competition and a default value for the date, which is set to the current date if not specified.

```sql
INSERT INTO Competition (compID, name, venue, date) VALUES
(101, 'HackathonX', 'Chennai Trade Center', '2025-09-01'),
(102, 'TechNova', 'IIT Madras', '2025-09-05'),
(103, 'CodeFest', 'Anna University', '2025-09-10'),
(104, 'InnoJam', 'Hyderabad Convention', '2025-09-15'),
(105, 'DevSprint', 'VIT Vellore', '2025-09-20'),
(106, 'RoboRace', 'NIT Trichy', '2025-09-25'),
(107, 'AlgoMania', 'Osmania University', '2025-09-30'),
(108, 'SmartCity', 'SRM University', '2025-10-02'),
(109, 'HackSphere', 'JNTU Hyderabad', '2025-10-07'),
(110, 'CodeStorm', 'BITS Hyderabad', '2025-10-12'),
(111, 'TechThon', 'PSG Tech', '2025-10-17'),
(112, 'CloudHack', 'IIT Madras', '2025-10-22'),
(113, 'CyberWarriors', 'Anna University', '2025-10-27'),
(114, 'DeepHack', 'IIIT Hyderabad', '2025-11-01'),
(115, 'RoboVision', 'NIT Warangal', '2025-11-06'),
(116, 'DataThon', 'IIT Hyderabad', '2025-11-11'),
(117, 'SecureCode', 'JNTU Kakinada', '2025-11-16'),
(118, 'AIJam', 'SRM Chennai', '2025-11-21'),
(119, 'MegaHack', 'Andhra University', '2025-11-26'),
(120, 'TechBlast', 'Chennai Trade Center', '2025-12-01');
```

Result Grid | Filter Rows: | Edit:

compID	name	venue	date
101	HackathonX	Chennai Trade Center	2025-09-01
102	TechNova	IIT Madras	2025-09-05
103	CodeFest	Anna University	2025-09-10
104	InnoJam	Hyderabad Convention	2025-09-15
105	DevSprint	VIT Vellore	2025-09-20
106	RoboRace	NIT Trichy	2025-09-25
107	AlgoMania	Osmania University	2025-09-30
108	SmartCity	SRM University	2025-10-02
109	HackSphere	JNTU Hyderabad	2025-10-07
110	CodeStorm	BITS Hyderabad	2025-10-12
111	TechThon	PSG Tech	2025-10-17
112	CloudHack	IIT Madras	2025-10-22
113	CyberWarr...	Anna University	2025-10-27
114	DeepHack	IIIT Hyderabad	2025-11-01
115	RoboVision	NIT Warangal	2025-11-06
116	DataThon	IIT Hyderabad	2025-11-11
117	SecureCode	JNTU Kakinada	2025-11-16
118	AIJam	SRM Chennai	2025-11-21
119	MegaHack	Andhra University	2025-11-26
120	TechBlast	Chennai Trade Center	2025-12-01
NULL	NULL	NULL	NULL

```
CREATE TABLE Organisers (
orgID INT PRIMARY KEY,
    comp INT NOT NULL,
    FOREIGN KEY (orgID) REFERENCES Participants(id) ON DELETE CASCADE,
    FOREIGN KEY (comp) REFERENCES Competition(compID) ON DELETE CASCADE
);
```

```
INSERT INTO Organisers (orgID, comp) VALUES
(6, 101),
(7, 102),
(8, 103),
(9, 104),
(10, 105),
(16, 106),
(17, 107),
(18, 108),
(19, 109),
(20, 110),
(1, 111),
(2, 112),
(3, 113),
(4, 114),
(5, 115),
(11, 116),
(12, 117),
(13, 118),
(14, 119),
(15, 120);
```

The Organisers table records which participant organizes which competition. It contains attributes **orgID and comp**, where orgID is linked to Participants and comp is linked to Competition through foreign keys.

Result Grid

orgID	comp
6	101
7	102
8	103
9	104
10	105
16	106
17	107
18	108
19	109
20	110
1	111
2	112
3	113
4	114
5	115
11	116
12	117
13	118
14	119
15	120
NULL	NULL

The Mentorship table represents a **many to many relationship between students and mentors**. It uses a **composite primary key of studentID and mentorID**. This allows each student to be guided by multiple mentors and each mentor to guide multiple students.

```sql
CREATE TABLE Mentorship (
    studentID INT NOT NULL,
    mentorID INT NOT NULL,
    PRIMARY KEY (studentID, mentorID),
    FOREIGN KEY (studentID) REFERENCES Student(studentID),
    FOREIGN KEY (mentorID) REFERENCES Mentor(MentorID)
);
```

```sql
INSERT INTO Mentorship (studentID, mentorID) VALUES
(1, 11), (2, 12), (3, 13), (4, 14), (5, 15),
(6, 16), (7, 17), (8, 18), (9, 19), (10, 20),
(1, 12), (2, 13), (3, 14), (4, 15), (5, 16),
(6, 17), (7, 18), (8, 19), (9, 20), (10, 11);
```

Result Grid | Filter Ro

studentID	mentorID
1	11
10	11
1	12
2	12
2	13
3	13
3	14
4	14
4	15
5	15
5	16
6	16
6	17
7	17
7	18
8	18
8	19
9	19
9	20
10	20
NULL	NULL

The MentorHierarchy table illustrates a **recursive relationship among mentors**. It contains attributes **seniorID and juniorID**, both referencing to Mentor table.

```
CREATE TABLE MentorHierarchy (
    seniorID INT,
    juniorID INT,
    PRIMARY KEY (seniorID, juniorID),
    FOREIGN KEY (seniorID) REFERENCES Mentor(mentorID),
    FOREIGN KEY (juniorID) REFERENCES Mentor(mentorID)
);

INSERT INTO MentorHierarchy (seniorID, juniorID) VALUES
(11, 12), (12, 13), (13, 14), (14, 15), (15, 16),
(16, 17), (17, 18), (18, 19), (19, 20), (20, 11),
(11, 13), (12, 14), (13, 15), (14, 16), (15, 17),
(16, 18), (17, 19), (18, 20), (19, 11), (20, 12);
```

Result Grid

seniorID	juniorID
19	11
20	11
11	12
20	12
11	13
12	13
12	14
13	14
13	15
14	15
14	16
15	16
15	17
16	17
16	18
17	18
17	19
18	19
18	20
19	20
NULL	NULL

```sql
CREATE TABLE ParticipatesIN (
studentId INT NOT NULL,
    compID INT NOT NULL,
    mentorID INT NOT NULL,
    PRIMARY KEY (studentID, compID, mentorID),
    role VARCHAR(20) DEFAULT "Contestant" CHECK (role in ('Contestant', 'TeamLead')),
    FOREIGN KEY (studentID) REFERENCES Student(studentID) ON DELETE CASCADE,
    FOREIGN KEY (compID) REFERENCES Competition(compID) ON DELETE CASCADE,
    FOREIGN KEY (mentorID) REFERENCES Mentor(mentorID) ON DELETE CASCADE
);
```

```sql
INSERT INTO ParticipatesIN (studentID, compID, mentorID, role) VALUES
(1, 101, 11, 'Contestant'),
(2, 102, 12, 'TeamLead'),
(3, 103, 13, 'Contestant'),
(4, 104, 14, 'TeamLead'),
(5, 105, 15, 'Contestant'),
(6, 106, 16, 'TeamLead'),
(7, 107, 17, 'Contestant'),
(8, 108, 18, 'TeamLead'),
(9, 109, 19, 'Contestant'),
(10, 110, 20, 'TeamLead'),
(1, 111, 12, 'Contestant'),
(2, 112, 13, 'Contestant'),
(3, 113, 14, 'Contestant'),
(4, 114, 15, 'Contestant'),
(5, 115, 16, 'Contestant'),
(6, 116, 17, 'Contestant'),
(7, 117, 18, 'Contestant'),
(8, 118, 19, 'Contestant'),
(9, 119, 20, 'Contestant'),
(10, 120, 11, 'Contestant');
```

The ParticipatesIN table shows the **ternary relationship between students, competitions, and mentors.** It has attributes **studentID, compID, mentorID, and role**. A constraint is on role to allow only the values ‘Contestant’ or ‘TeamLead’.

Result Grid | Filter Rows:

studentId	compID	mentorID	role
1	101	11	Contestant
1	111	12	Contestant
2	102	12	TeamLead
2	112	13	Contestant
3	103	13	Contestant
3	113	14	Contestant
4	104	14	TeamLead
4	114	15	Contestant
5	105	15	Contestant
5	115	16	Contestant
6	106	16	TeamLead
6	116	17	Contestant
7	107	17	Contestant
7	117	18	Contestant
8	108	18	TeamLead
8	118	19	Contestant
9	109	19	Contestant
9	119	20	Contestant
10	110	20	TeamLead
10	120	11	Contestant
NULL	NULL	NULL	NULL

```sql
CREATE TABLE Awards (
    studentID INT,
    compID INT,
    PRIMARY KEY (compID, studentID),
    position INT CHECK (position IN (1, 2, 3)),
    FOREIGN KEY (studentID) REFERENCES Student(studentID) ON DELETE CASCADE,
    FOREIGN KEY (compID) REFERENCES Competition(compID) ON DELETE CASCADE
);
```

```sql
INSERT INTO Awards (studentID, compID, position) VALUES
(1, 101, 1),
(2, 102, 2),
(3, 103, 3),
(4, 104, 1),
(5, 105, 2),
(6, 106, 3),
(7, 107, 1),
(8, 108, 2),
(9, 109, 3),
(10, 110, 1),
(1, 111, 2),
(2, 112, 3),
(3, 113, 1),
(4, 114, 2),
(5, 115, 3),
(6, 116, 1),
(7, 117, 2),
(8, 118, 3),
(9, 119, 1),
(10, 120, 2);
```

Result Grid | Filter Rows:

studentID	compID	position
1	101	1
2	102	2
3	103	3
4	104	1
5	105	2
6	106	3
7	107	1
8	108	2
9	109	3
10	110	1
1	111	2
2	112	3
3	113	1
4	114	2
5	115	3
6	116	1
7	117	2
8	118	3
9	119	1
10	120	2
NULL	NULL	NULL

The Awards table stores competition results by mapping students to the competitions they took part in along with their positions. Its attributes are **studentID, compID, and position**. A constraint ensures that the position value can only be 1, 2, or 3.

Aggregate functions:

```
224    -- Count how many students are in each department
225 •  SELECT dept, COUNT(*) AS total_students
226    FROM Student
227    GROUP BY dept;
```

Result Grid | Filter Rows: | Export: | Wrap Cell (

dept	total_students
CSE	2
ECE	2
EEE	2
MECH	2
IT	1
Civil	1

COUNT function was applied to find the number of students in each department

```
229    -- Find the average experience of mentors
230 •  SELECT AVG(exp) AS avg_exp
231    FROM Mentor;
```

Result Grid | Filter Rows: | Export: | Wrap

avg_exp
7.9000

AVG function calculated the avg years of experience.

```
234    -- Get the maximum and minimum experience among mentors
235 •  SELECT MAX(exp) AS max_exp,
236           MIN(exp) AS min_exp
237    FROM Mentor;
```

MAX and MIN functions identified the highest and lowest experience.

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

max_exp	min_exp
15	3

```
238    -- Find how many competitions each student participated in
239 •  SELECT studentID, COUNT(compID) AS comp_attended
240    FROM ParticipatesIN
241    GROUP BY studentID;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

studentID	comp_attended
1	2
2	2
3	2
4	2
5	2
6	2
7	2
8	2
9	2
10	2

COUNT with GROUP BY was used to check how many competitions each student participated in

GROUP BY and HAVING:

```
249     -- Mentors with average student count >= 2
250     SELECT mentorID, COUNT(studentID) AS total_students
251     FROM Mentorship
252     GROUP BY mentorID
253     HAVING COUNT(studentID) >= 2;
254
```

Result Grid | Filter Rows: | Export: | Wrap Cell Con

mentorID	total_students
11	2
12	2
13	2
14	2
15	2
16	2
17	2
18	2
19	2
20	2

The **GROUP BY and HAVING** clauses are used to **filter results based on conditions**. The query shows mentors who guide at least two students, from mentorship.

```
261 •  CREATE USER 'user1'@'localhost' IDENTIFIED BY 'user123';
262 •  GRANT SELECT ON UniHackathon.* TO 'user1'@'localhost';
263 •  FLUSH PRIVILEGES;
```

A new user account was created in using the terminal. The user is granted with **SELECT privilege**, which means they can view data but cannot make changes. After logging in as user1, SELECT queries were executed.

```
C:\Users\Bhaavya>mysql -u user1 -p
'mysql' is not recognized as an internal or external command,
operable program or batch file.

C:\Users\Bhaavya>cd "C:\Program Files\MySQL\MySQL Server 8.0\bin"

C:\Program Files\MySQL\MySQL Server 8.0\bin>mysql -u user1 -p
Enter password: *******
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 10
Server version: 8.0.42 MySQL Community Server - GPL

Copyright (c) 2000, 2025, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql>
```

Command Prompt - mysql -u

```
mysql> USE UniHackathon;
Database changed
mysql> SHOW TABLES;
+-------------------------+
| Tables_in_unihackathon |
+-------------------------+
| awards                  |
| competition             |
| mentor                  |
| mentorhierarchy         |
| mentorship              |
| organisers              |
| participants            |
| participatesin          |
| student                 |
+-------------------------+
9 rows in set (0.04 sec)

mysql> -- Count how many students are in each department
mysql> SELECT dept, COUNT(*)
    -> FROM Student
    -> GROUP BY dept;
+-------+----------+
| dept  | COUNT(*) |
+-------+----------+
| CSE   |        2 |
| ECE   |        2 |
| EEE   |        2 |
| MECH  |        2 |
| IT    |        1 |
| Civil |        1 |
+-------+----------+
6 rows in set (0.00 sec)

mysql>
mysql> -- Average mentor experience
mysql> SELECT AVG(exp) AS avg_experience
    -> FROM Mentor;
+----------------+
| avg_experience |
+----------------+
|         7.9000 |
+----------------+
1 row in set (0.00 sec)
```